# ELECTION CLASS UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 01**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: obtainFileNameTest is the name of the test. This test makes sure that the input of the file name is the correct file name.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass ___X__ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of the Ballot program**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of an election | | | | |
| 3 | Stream a file name | | | | |
| 4 | Obtain the actual file name | The input file from the username | In this test case, the input from the user will be SystemTestCPL1.csv | SystemTestCPL1.csv" | |
| 5. | Check that the end of the file name contains .csv | | | | |
| | | | | | |

**Post condition(s) for Test:**

The system will be able to start the process knowing that an existing csv file of votes is created.

---

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 02** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: parseFileTest is the name of the test. This test makes sure that the information from the file is correctly parsed and the correct information is placed within each and every class.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass ___X__ Fail_____**

**Preconditions for Test:**
**A successful make and ./Test of the Ballot program. The test obtainFileNameTest has been passed.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of an election with the file name (One for the CPL algorithm and one for the IR algorithm) | | | | |
| 3 | Create a string name for the file name (SystemTESTCPL1.csv | | | | |
| 4 | Create a string date to represent the date(12-12-2022) | | | [redacted] | |
| 5. | Create a stringstream s with the date | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 6. | Create an int to represent the result of the election.parseFile with the filename and the stringstream | res | 0 | 0 | |

**Post condition(s) for Test:**

The system will be able to parse the information from the file and be able to create and run an algorithm based on the data of the file.

## Project Name: Project 1: Voting System — Team# 8

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 03**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ElectionTest and the name of the test is the obtainDate. This test ensures that the date by the user is entered in the correct form.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of Election. The user must be prompted to enter the date when prompted by the terminal**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of CPLAlgorithm() | | | | |
| 3 | Create a string date to represent the date(12-12-2022) | | | | |
| 4 | Create a stringstream s with the date | | | | |
| 5 | Create a string to represent the election.obtainDate(s) | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 6 | Test the equality of the actual string with the date | actual = election.obtainDate(s) | 12-12-2022 | 12-12-2022 | |

**Post condition(s) for Test:**

The system will be able to store the date of the election which will then later be used in the naming convention for the AuditFile

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 04** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ElectionTest and the name of the test is the IRParsing This test ensures that the helper function for parsing the IR file works as intended. The test makes sure the function works for the specified file format specified in the SRS.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X_____ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of Election.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of Election() | | | | |
| 3 | Create a string of ballot information ballotStr | | | | |
| 4 | Create four different instance Canindate all with different names | | | | |
| 5 | Create a vector of Canindate (canindates) and then push back all of the candidate created in Step #4 onto the | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | vector | | | | |
| 6 | Create an instance of BallotIr that is created from election.Parsing(ballotStr, candidate.size(), candidates) | | | | |
| 7 | Test the equality of the actual candidates from the ballot and the names created in Step#2 | ballot→getCandidate(0)<br>ballot→getCandidate(1)<br>ballot→getCandidate(2)<br>ballot→getCandidate(3) | "jessica"<br>"joe"<br>"sarah"<br>"bob" | "jessica"<br>"joe"<br>"sarah"<br>"bob" | |

**Post condition(s) for Test:**

The system will be able successfuly parse information from an IR input ballot file and be able to to run the algorithm successfully

## Project Name: Project 1: Voting System — Team# 8

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 05**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ElectionTest and the name of the test is the CPLParsingParties This test ensures that the helper function for parsing the CPL file works as intended. The test correctly parses the party successfully.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Election.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of Election() | | | | |
| 3 | Create a string of ballot information ballotStr | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 4 | Create a vector of Party* that is initialized in election.CPLParsingParties(partystr) | | | | |
| 7 | Test the equality of the actual party names with the parties from the ballotStr | parties[0] → getName()<br>parties[1] →getName()<br>parties[2] →getName() | "Democrats"<br>"Republicans"<br>"Independent | Democrats"<br>"Republicans"<br>"Independent | |

**Post condition(s) for Test:**

The system will be able successfully parse party information from a CPL input ballot file. The information parsed from this function will be usefully when running the CPL Algorithm

---

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 06** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ElectionTest and the name of the test is the CPLParsingCandidates. This test ensures that the helper function for parsing the CPL file works as intended. The test correctly parses the candidates from the input file successfully.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no __**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Election.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 2 | Create an instance of Election() | | | | |
| 3 | Create a string of candidate information (canindate string) | | | | |
| 4 | Create a vector of string that is initialized with election.CPLParsingCanindates(canindateString) | | | | |
| 7 | Test the equality of the actual candidate names with the candidates from the canindatestr | candidates[0]<br>candidates[1]<br>candidates[2]<br>candidates[3] | “Bob”<br>“Joe”<br>“Jessica”<br>“Sarah” | “Bob”<br>“Joe”<br>“Jessica”<br>“Sarah” | |

**Post condition(s) for Test:**

The system will be able successfully parse candidate information from a CPL input ballot file. The information parsed from this function will be usefully when running the CPL Algorithm

# IRALGORITHM CLASS UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 07** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetSetBallot**

**This test ensure that the getter and setter of the IRAlgorithm for ballots work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the 2 vector of string b1 and b2 | N/A | N/A | N/A | |
| 3 | add the string “Ballot” to b1 and “Ballot2” to b2 | N/A | N/A | N/A | |
| 4 | create 2 BallotIR object | N/A | N/A | N/a | |
| 5 | create a ballot IR vector ballots | | | | |
| 6 | add 2 ballotIR to the ballots vector | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | check to see if the first index of algo get ballot work | alg.getBallot().at(0)->getCandidate(0) == "Ballot" | true | true | |
| 9 | check to see if the second index of algo get ballot work | alg.getBallot().at(0)->getCandidate(0) == "Ballot2" | true | true | |
| 10 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to set the ballots and get the ballots correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 08**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetSetCandidates**

**This test ensure that the getter and setter of the IRAlgorithm for Candidates work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create 2 candidate object | | | | |
| 3 | create vector contain candidate | | | | |
| 4 | add 2 candidates to the vector candidates | | | | |
| 5 | create IRAlgorithm object | | | | |
| 6 | set candidates in IRAlgoirthm | alg.getCandidates().at(0).getCandidateName() == "Candidate1" | true | true | |
| 7 | check getter in IRAlgorithm | alg.getCandidates().at(0).getCandidateName() == "Candidate1" | true | true | |
| 8 | | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 9 | | | | | |
| 10 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to set the Candidate and get the Candidate correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 09**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetSetNumOfCandidates**

**This test ensure that the getter and setter of the IRAlgorithm for number of Candidates work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create IRAlgorithm object | | | | |
| 3 | set IRAlgorithm number of candidate by 10 people | | | | |
| 4 | call the getNumberOfCandidates() | alg.getNumberOfCandidates() == 10 | true | true | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to set the number of Candidates and get the banumber of Candidates llots correctly

---

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 10**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetSetNumOfBallots**

**This test ensure that the getter and setter of the IRAlgorithm for number of Ballots work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X_____ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | create the IRAlgorithm object | | | | |
| 3 | set IRAlgorithm number of candidate by 10 ballots | | | | |
| 4 | call the getNumberOfBallots() | alg.getNumberOfBallots() == 10 | True | True | |
| 5 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to set the number of Ballots and get the banumber of Ballots correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 11**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetStepsTest**

**This test ensure that the getter of the IRAlgorithm for the step correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes _X_ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the IRAlgorithm object | | | | |
| 3 | create the vector of string contain steps | | | | |
| 4 | call the getSteps() | alg.getSteps() == steps | True | True | |
| 5 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to get the steps correct

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 12** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is GetSetWinnerTest**

**This test ensure that the getter and setter of the IRAlgorithm for winner correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the IRAlgorithm alg object | | | | |
| 3 | set the winner in the alg | | | | |
| 4 | call the getWinner() | alg.getWinner() == "winner" | True | True | |
| 5 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to get the winner correct

## Project Name: Project 1: Voting System

## Team# 8

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 13**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is breakTieTest**

**This test ensure that the breaktieTest of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | alg.getSteps() == steps | True | True | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | call breakTie() in IRAlgorihtm object | algpara.breakTie(candidates).getCandidateName() == "x" \|\| "y" \|\| "z" | True | True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to break the tie

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 14**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set**

**IRAlgorithmTest and the name of the test is countBallotsTest**

**This test ensure that the countBallot() of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | call countBallot() in IRAlgorihtm object | | | | |
| 9 | compare the total vote for each Candidates in IRAlgorithm | algpara.getCandidates().at(0).getCandidateTotalVote() == 2<br>algpara.getCandidates().at(1).getCandidateTotalVote() == 1<br>algpara.getCandidates().at(2).getCandidateTotalVote() == 0 | True<br>True<br>True | True<br>True<br>True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to count Ballot correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 15**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is redistributeBallotsTest**

**This test ensure that the redistributeBallots() of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X___ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | 3 | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | call countBallots() | | | | |
| 8 | call redistributeBallots(candidatex) | | | | |
| 9 | compare the total vote for each Candidates in IRAlgorithm | algpara.getCandidates().at(0).getCandidateTotalVote() == 2<br>algpara.getCandidates().at(1).getCandidateTotalVote() == 1<br>algpara.getCandidates().at(2).getCandidateTotalVote() == 0 | True<br>True<br>True | True<br>True<br>True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to redistribute Ballot correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 16**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is pickPopularCandidateTest**

**This test ensure that the pickPopularCandidate() of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | call countBallots() | | | | |
| 8 | call pickPopularCandidate() and get the candidateName | | | | |
| 9 | compare the name of the popular candidate | algpara.pickPopularCandidate().getCandidateName() == "x" | True | True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to pick the popular Candidate correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 17** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is getLeastVoteCandidateTest**

**This test ensure that the getLeastVoteCandidate() of the**

**IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object | | | | |
| 8 | call countBallots() | | | | |
| 8 | call getLeastVoteCandidate() and get the candidateName | | | | |
| 9 | compare the name of the least vote candidate | algpara.getLeastVoteCandidate().getCandidateName() == "z", true | True | True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to pick the least vote Candidate correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 18**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is getMajorityTest**

**This test ensure that the getMajority() of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 7 | create IRAlgorithm object | | | | |
| 8 | call countBallots() | | | | |
| 8 | call getMajority() and get the winner | | | | |
| 9 | compare the name of the winnning candidate | algpara.getWinner() == "x" | True | True | |

**Post condition(s) for Test:**

The IRAlgorithm will be able get majority Candidate correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 19**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is emptyContructorTest**

**This test ensure that the default constructor of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X_____ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create IRAlgorithm object | | | | |
| 3 | check the number of candidate in empty constructor | alg.getNumberOfCandidates() == 0 | true | true | |
| 4 | check the number of ballots in empty constructor | alg.getNumberOfBallots() == 0 | true | true | |
| 5 | | | | | |
| 6 | | | | | |
| 7 | | | | | |
| 8 | | | | | |
| 8 | | | | | |
| 9 | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to create default constructor correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 20** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is contructorParameterTest**

**This test ensure that the constructor parameter of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object filled with parameters | | | | |
| 8 | check the ballots | algpara.getBallot() == ballots | True | True | |
| 9 | check the number of candidates | algpara.getNumberOfCandidates() == numOfCandidates | True | True | |
| 10 | check all 3 candidates name | algpara.getCandidates().at(0).getCandidateName() == "x"<br>algpara.getCandidates().at(1).getCandidateName() == "y"<br>algpara.getCandidates().at(2).getCandidateName() == "z" | True<br>True<br>True | True<br>True<br>True | |
| 11 | check number of ballot | algpara.getNumberOfBallots() == numOfBallots | True | True | |
| | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to create constructor with parameters correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 21**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is RunIRTest1**

**This test ensure that the runIR of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object filled with parameters | | | | |
| 8 | call the runIR() function | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 9 | compare the return result | result == "Completed" | True | True | |
| 11 | compare the winner | algpara.getWinner() == "x" | True | True | |
| | | | | | |

**Post condition(s) for Test:**

The IRAlgorithm will be able to run IR algorithm with parameters correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 22**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set IRAlgorithmTest and the name of the test is RunIRTest2**

**This test ensure that the runIR of the IRAlgorithm work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X_____ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step | Test Step | Test | Expected | Actual | |
|---|---|---|---|---|---|

| # | Description | Data | Result | Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | 3 vector of string filled with characters with different order | | | | |
| 3 | create 3 ballotIR filled where each ballotIR filled with each vector of string | | | | |
| 4 | create a vector contain BallotIR and filled them with the 3 ballotIR | | | | |
| 5 | create 3 different candidates and filled 3 candidates to a vector of Candidates. | | | | |
| 6 | set number of ballot to be equal 3 | | | | |
| 7 | create IRAlgorithm object filled with parameters | | | | |
| 8 | call the runIR() function | | | | |
| 9 | compare the return result | result == "Completed" | True | True | |
| 11 | compare the winner | algpara.getWinner() == "y" | True | True | |
| | | | | | |

# RESULT CLASS UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 23** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ResultTest and the name of the test is emptyContructorTest**

**This test ensure that the default constructor of the Result work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of ResultTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the Result object | | | | |
| 3 | call the getTypeOfVoting() and compare the expected output | myResult.getTypeOfVoting() | “ “ | “ “ | |
| 4 | call the getNumberOfVote() and compare the expected output | myResult.getNumberOfVote() | 0 | 0 | |
| 5 | call the getWinner() and compare the expected output | myResult.getWinner() | “ “ | “ “ | |

**Post condition(s) for Test:**

The Result will be able to create default constructor correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 24** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ResultTest and**

**the name of the test is gettersetterTest**

**This test ensure that the getter and setter of the Result work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of ResultTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the Result object for IR with 300 votes, and the winner name is Dorthy | | | | |
| 3 | call the getTypeOfVoting() and compare the expected output | myResult.getTypeOfVoting() | "IR" | "IR" | |
| 4 | call the getNumberOfVote() and compare the expected output | myResult.getNumberOfVote() | 300 | 300 | |
| 5 | call the getWinner() and compare the expected output | myResult.getWinner() | "Dorthy" | "Dorthy" | |
| 6 | set the type of voting to CPL | | | | |
| 7 | set the number of voting to 4546 | | | | |
| 8 | set the winner to be Larrry | | | | |
| 9 | call the getTypeOfVoting() and compare the expected output | myResult.getTypeOfVoting() | "CPL" | "CPL" | |
| 10 | call the getNumberOfVote() and compare the expected output | myResult.getNumberOfVote() | 4546 | 4546 | |
| 11 | call the getWinner() and compare the expected output | myResult.getWinner() | "Larry" | "Larry" | |

**Post condition(s) for Test:**

The Result will be able to to have the getter and setter work correctly

# AUDITFILE CLASS UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 25**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set AuditTest and the name of the test is emptyConstructor**

**This test ensure that the default constructor of the AuditFile work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of AuditTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create the AuditFile object | | | | |
| 3 | compare the voting type | audit.getVotingType() | “ “ | “ “ | |

| | | | | | |
|---|---|---|---|---|---|
| 4 | compare the number of Candidate | audit.getBallotNum() | 0 | 0 | |
| 5 | compare the number of Ballots | audit.getNumberCandidates() | 0 | 0 | |
| 6 | compare the winner | audit.getWinner() | “ “ | “ “ | |

**Post condition(s) for Test:**

The AuditFile will be able to to have default constructor work correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 26**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set AuditTest and the name of the test is CPLConstructor**

**This test ensure that the CPL constructor of the AuditFile work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of AuditTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create a party vector contain 2 party | | | | |
| 3 | create a string vector contain 2 steps | | | | |
| 4 | create the CPL constructor with parameters | | | | |
| 5 | compare the voting type | audit.getVotingType() | "CPL" | "CPL" | |
| 6 | compare the number of ballots | audit.getBallotNum() | 2 | 2 | |
| 7 | compare the step at index 0 | audit.getSteps().at(0) | “step1” | “step1” | |
| 8 | compare the step at index 1 | audit.getSteps().at(1) | “step2” | “step2” | |
| 9 | compare the winner | audit.getWinner() | “Democratic” | “Democratic” | |

**Post condition(s) for Test:**

The AuditFile will be able to to have CPL constructor work correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 27** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set AuditTest and the name of the test is CPLConstructor**

**This test ensure that the IR constructor of the AuditFile work correctly**

**Automated: yes_X__ no ___** **Indicate where are you storing the tests (what file) and the**

**name of the method/functions being used.**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of AuditTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create a candidate vector contain 2 Candidates | | | | |
| 3 | create a string vector contain 2 steps | | | | |
| 4 | create the CPL constructor with parameters | | | | |
| 5 | compare the voting type | audit.getVotingType() | "IR" | "IR" | |
| 6 | compare the number of ballots | audit.getBallotNum() | 2 | 2 | |
| 7 | compare the candidate at index 0 | audit.getCandidates().at(0).getCandidateName() | "Dorthy" | "Dorthy" | |
| 8 | compare the candidate at index 1 | audit.getCandidates().at(1).getCandidateName() | "Larry" | "Larry" | |
| 9 | compare the step at index 0 | audit.getSteps().at(0) | “step1” | “step1” | |
| 10 | compare the step at index 1 | audit.getSteps().at(1) | “step2” | “step2” | |
| 11 | compare the winner | audit.getWinner() | “Dorthy” | “Dorthy” | |

**Post condition(s) for Test:**

The AuditFile will be able to to have CPL constructor work correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 28** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set AuditTest and the name of the test is createAuditFileCPLTest**

**This test ensure that the default constructor of the AuditFile work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of AuditTest**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | create a candidate vector contain 2 Candidates | | | | |
| 2 | create a string vector contain 2 steps | | | | |
| 3 | create the CPL constructor with parameters | | | | |
| 4 | compare the voting type | audit.getVotingType() | "CPL" | "CPL" | |
| 5 | compare the number of ballots | audit.getBallotNum() | 2 | 2 | |
| 8 | compare the step at index 0 | audit.getSteps().at(0) | “step1” | “step1” | |
| 9 | compare the step at index 1 | audit.getSteps().at(1) | “step2” | “step2” | |
| 10 | compare the winner | audit.getWinner() | “Democratic” | “Democratic” | |
| 11 | call create_AuditFIleCPL() | | | | |
| 12 | compare the filename | audit.create_AuditFileCPL() | "CPL12-12-2022.txt" | "CPL12-12-2022.txt" | |

**Post condition(s) for Test:**

The AuditFile will be able to create Auditfile CPL work correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 29**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set AuditTest and the name of the test is createAuditFileIRTest**

**This test ensure that the IR constructor of the AuditFile work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail _____**

**Preconditions for Test:**
**A successful make test and ./Test of AuditTest**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create a candidate vector contain 2 Candidates | | | | |
| 3 | create a string vector contain 2 steps | | | | |
| 4 | create the CPL constructor with parameters | | | | |
| 5 | compare the voting type | audit.getVotingType() | "IR" | "IR" | |

| | | | | | |
|---|---|---|---|---|---|
| 6 | compare the number of ballots | audit.getBallotNum() | 2 | 2 | |
| 7 | compare the candidate at index 0 | audit.getCandidates().at(0).getCandidateName() | "Dorthy" | "Dorthy" | |
| 8 | compare the candidate at index 1 | audit.getCandidates().at(1).getCandidateName() | "Larry" | "Larry" | |
| 9 | compare the step at index 0 | audit.getSteps().at(0) | “step1” | “step1” | |
| 10 | compare the step at index 1 | audit.getSteps().at(1) | “step2” | “step2” | |
| 11 | compare the winner | audit.getWinner() | “Dorthy” | “Dorthy” | |
| 12 | call the function create_AuditFileIR() | | | | |
| 13 | compare the file name | audit.create_AuditFileIR() | "IR12-12-2022.txt" | "IR12-12-2022.txt" | |

**Post condition(s) for Test:**

The AuditFile will be able to to have create Audit File IR work correctly

# BALLOTCPL CLASS UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 30**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ballotCPLTest is and the name of the test is the emptyConstructor. The whole purpose of the test is to make sure that an instance of a ballotCPL is created**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass ___X__ Fail_____**

**Preconditions for Test:**
**A successful make and ./run of the Ballot program. The file input is correct and the file name ends with .csv**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an expected string that contains the expected party name | | string partyName = "democrat"; | | |
| 3 | Create a pointer to the party class that stores the party Infromation from step #2 | Party *party = new Party(partyName); | | | |
| 4 | Create an instance BallotCPL with the pointer from step #3 and the string from step #2. | BallotCPL ballot = BallotCPL(partyName, party); | | | |
| 5 | Expect true with the party of the instance of the ballot equivalent to the party | ballot.getParty() | partyName (democrat) | demo | |

**Post condition(s) for Test:**

Successful creation of a Ballot constructor.

---

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 31**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set ballotCPLTest is and the name of the test is the "getParty". The whole purpose of the test is to make sure that the correct party is associated with the correct CPL.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass ___X__ Fail_____**

**Preconditions for Test:**
**A successful make and ./run of the Ballot program. A successful instance of a BallotCPL is created either with the successful creation of a BallotCPL.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a string of a party name | | | | string partyName = "republican"; |
| 3 | Create a pointer to the party class with the Party constructor and the party string name | | | | Party *party = new Party(partyName) |
| 4 | Increment the votes | | | | party->incrementVote(); |
| 5 | Create a vector of string that contains some candidates. | | | | vector<string> candidates = {"Bob", "Joe", "Sarah"}; |
| 6 | Set the party candidates to the vector of string | | | | party->setCandidates(candidates); |
| 7 | Create an instance BallotCPL with the pointer from step #3 and the string from step #2. | | 0 | 0 | BallotCPL ballot = BallotCPL(partyName, party); |
| 8. | Test the party of the ballot is the same as the party string created | ballot.getParty() | party(republican) | republican | |

**Post condition(s) for Test:**

The getParty function inside of the ballotCPL class works

**CANDIDATE CLASS TEST**

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 32**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CanindateTest is and the name of the test is the "CanindateTest". The whole purpose of the test is to make sure that the empty constructor contains the default values. It also checks to see if some of the getters and setters are working for the class. There are also some tests of the different functions.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make and ./run of the Ballot program. A successful instance of a BallotCPL is created either with the successful creation of a BallotCPL.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of an empty Constructor | | | | |
| 3 | Test the equality of each of the getters of the candidate class to make sure the return default data | candidate.getBallots().empty<br>candidate.getActive()<br>candidate.getCandidateTotalVotes<br>candidate.getCandidateName() | true<br>true<br>0<br>" " | true<br>true<br>0<br>" " | |
| 4 | Set the candidate activity to false | | | | |
| 5 | Test the activity of the canindate. | candidate.getActive() (should be false) | false | false | |
| 6 | Create two different vector of string and push back a Test string into each vector. | | | | |
| 7 | Create two different BallotIR references with the BallotIR constructor (ballot and ballotIR) and the two different string names created in Step #6 (Test and Test1) | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 8. | Append the ballot to the canindate | | | | |
| 9. | Test that the ballots for the candidate contain the correct strings | candidate.getBallots().at(0)->getCandidate(0) | Test | Test | |
| 10 | Test the total vote of candidate | candidate.getCandidateTotalVote() | 1 | 1 | |
| 11 | Append the ballot1 to the candidate | | | | |
| 12 | Test that the ballots for the candidate contain the correct strings | candidate.getBallots().at(1)->getCandidate(0) | Test1 | Test1 | |
| 13 | Test the total vote of candidate | candidate.getCandidateTotalVote() | 2 | 2 | |
| 14 | Create an instance of Canindate with a Test String | | | | |
| 15. | Test the equality of each of the getters of the candidate class to make sure the return default data | candidate2.getBallots().empty<br>candidate2.getActive()<br>candidate2.getCandidateTotalVotes<br>candidat2.getCandidateName() | true<br>true<br>0<br>“ “ | true<br>true<br>0<br>“ “ | |
| 16 | Set the candidate2 activity to false | | | | |
| 17 | Test the activity of the canindate. | candidate.getActive() (should be false) | false | false | |

**Post condition(s) for Test:**

The candidate class and its functions are working correctly

# BALLOTIR UNIT TEST CLASS

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 33**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set BallotIR and the name of the test is the fullTest. This test ensures that all of the BallotIR functions work properly. BallotIR is simple enough that we can cover everything in one test.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
All expect statements run properly without failing

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of the BallotIR | | | | |
| 3 | Check if the rank of the ballot is 0 | | 0 | 0 | |
| 4 | Increment the rank | | | | |
| 5 | Check if the rank of the ballot is 1 | | 1 | 1 | |

| | | | | | |
|---|---|---|---|---|---|
| 6 | check that the ballot is valid | | true | true | |
| 7 | set the ballot to invalid | | | | |
| 8 | check if the ballot is not valid | | false | false | |
| 9 | Create a vector of strings for candidates | | | | |
| 10 | Check if candidates are the same | | {“test”, “test1”} | {“test”, “test1”} | |
| 11 | Create ballot2 | | | | |
| 12 | check if the rank 0 | | 0 | 0 | |
| 13 | increment rank | | | | |
| 14 | check if rank is 1 | | 1 | 1 | |
| 15 | check if ballot is valid | | true | true | |
| 16 | set ballot to invalid | | | | |
| 17 | check if ballot is invalid | | false | false | |

**Post condition(s) for Test:**

The BallotIR class functions fully

# CPLALGORITHM UNIT TEST CLASS

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 33**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the emptyConstructor. This test ensures that the CPLAlgorith empty constructor sets the private variables correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of the CPLAlgorithm | | | | |
| 3 | Create a vector of BallotCPL (ballots) | | | | |
| 4 | Create a vector of string (steps) | | | | |
| 5 | Create a vector of Party* (parties) | | | | |
| 6 | Test that the constructor contains all of the default values for the empty contructor | cp1.getBallots().size()<br>cp1.getNumberOfParties()<br>cp1.getParties()<br>cp1.getNumberOfSeats()<br>cp1.getNumberOfBallots()<br>cp1.getSteps() | 0<br>0<br>empty vector of BallotIR<br>0<br>0<br>empty vector of string | 0<br>0<br>empty vector of BallotIR<br>0<br>0<br>empty vector of string | |

**Post condition(s) for Test:**

The CPLAlgorithm works correctly for the empty constructor

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 34** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the constructor . This test ensures that the CPLAlgorith constructor sets the private variables correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a vector of BallotCPL (ballots) | | | | |
| 3 | Create a string name that contains a expect party name (partyName1) | | | | |
| 4 | Create a Party instance with the string created in step #3 | | | | |
| 5 | Add the Party instance from Step #4 to the vector in Step #2 | | | | |
| 6 | Initialize a variable tot he integer 1 | | | | |
| 7 | Create a vector of Party* that only contains the party created in step #3 | | | | |
| 8 | Create and initialize two different integer variables to 1 (numberOfSeats and numberOfBallots) | | | | |
| 9 | Create a vector of string called steps | | | | |
| 10 | Create an instance of CPLAlgorithm with the data structures created from the steps above | | | | |
| 11 | Test the equality of the default private member variables | cpl.getBallots()[0].getParty()<br>cpl.getNumberOfParties()<br>cp1.getParties()<br>cp1.getNumberOfSeats()<br>cp1.getNumberOfBallots()<br>cp1.getSteps() | Party<br>1<br>vector<Parties*><br>1<br>1<br>vector<string> | Party<br>1<br>vector<Parties*><br>1<br>1<br>vector<string> | |

**Post condition(s) for Test:**

The CPLAlgorithm works correctly for the overridden constructor

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 35**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the countBallots. This test ensures that the CPLAlgorithm correctly counts the number of ballot for each candidate and determines how many seats a candidate deserves based on their votes.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a vector of Party* (parties) | | | | |
| 3 | Create three strings with the different partyName (partyName1, partyName2, partyName3) | | | | |
| 4 | Create three different Party* | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | with the using the respective strings from Step#2 for the constructor (party1, party2, party3) | | | | |
| 5 | Add all the parties to the vector created in Step #2 | | | | |
| 6 | Create a vector of BallotCPL (ballots) | | | | |
| 7 | Create multiple instances of BallotCPL with their party string and party Name from Step #2 and Step#3 | | | | |
| 8 | Add all of the instances you have created from Step #7 to the vector created in step #6 | | | | |
| 9 | Create a CPL algorithm instance with ballots, 3, parties, -1, and 6 | | | | |
| 10 | Call the count ballots function | | | | |
| 11 | Test the equality of the parties remaining votes | parties[0] → getRemainingVotes()<br>parties[1] → getRemainingVotes()<br>parties[2] →getRemainingVotes() | 1<br>2<br>3 | 1<br>2<br>3 | |

**Post condition(s) for Test:**

The CPLAlgorithm countBallots function works as intended

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 36**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the countBallots. This test ensures that the CPLAlgorithm correctly calculates the quota for the election baked on the number of candidates and the number of seats.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of CPLAlgorithm() | | | | |
| 5 | Test the equality of certain calculateQuota parameters and the actual output | cpl.calculateQuota(8,3)<br>cpl.calculateQuota(9,3)<br>cpl.calculateQuota(10,3) | 2<br>3<br>3 | 2<br>3<br>3 | |

**Post condition(s) for Test:**

The CPLAlgorithm calculate function works as intended

## Project Name: Project 1: Voting System Team# 8

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 37**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the calculateSeats. This test ensures that the CPLAlgorithm correctly calculates the seats for the CPL election**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | Create an instance of CPLAlgorithm() | | | | |
| 5 | Test the equality of certain calculateSeats parameters and the actual output | cpl.calculateSeats(8,3)<br>cpl.calculateSeats(9,3)<br>cpl.calculateSeats(10,3) | 2<br>3<br>3 | 2<br>3<br>3 | |

**Post condition(s) for Test:**

The CPLAlgorithm calculate function works as intended

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 38**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the calculateRemainderVotes. This test ensures that the CPLAlgorithm correctly calculates the remainder number of votes for the CPL election**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|

| | | | | | |
|---|---|---|---|---|---|
| 2 | Create an instance of CPLAlgorithm() | | | | |
| 3 | Test the equality of certain calculateRemainderVotes parameters and the actual output | cpl.calculateRemainderVotes(8,3)<br>cpl.calculateRemainderVotes(9,3)<br>cpl.calculateRemainderVotes(10,3) | 2<br>0<br>1 | 2<br>0<br>1 | |

**Post condition(s) for Test:**

The CPLAlgorithm calculateRemainderVotes function works as intended

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 39**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the runCPL. This test ensures that the CPLAlgorithm correctly runs the CPL function and makes sure that the function correctly works with CPL>**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of CPLAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a vector of Party* (parties) | | | | |
| 3 | Create three strings with the | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | different partyName (partyName1, partyName2, partyName3) | | | | |
| 4 | Create three different Party* with the using the respective strings from Step#2 for the constructor (party1, party2, party3) | | | | |
| 5 | Add all the parties to the vector created in Step #2 | | | | |
| 6 | Create a vector of BallotCPL (ballots) | | | | |
| 7 | Create multiple instances of BallotCPL with their party string and party Name from Step #2 and Step#3 | | | | |
| 8 | Add all of the instances you have created from Step #7 to the vector created in step #6 | | | | |
| 9 | Create a CPL algorithm instance with ballots, 3, parties, 2, and 6 | | | | |
| 10 | Call the runCPL function | | | | |
| 11 | Test the equality of the parties remaining votes | parties[0] → getSeatsReceived ()<br>parties[1] → getSeatsReceived()<br>parties[2] → getSeatsReceived() | 0<br>1<br>1 | 0<br>1<br>1 | |

**Post condition(s) for Test:**

The CPLAlgorithm runCPL function works as intended

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 40** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the breakTie. This test ensures that the CPLAlgorithm correctly runs the CPL function and makes sure that the function correctly determines the winner of a tie in the most fair way possible**

**Automated: yes_X__ no ___**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor. There needs to be an instance of a tie for this function test to work.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an instance of the CPLAlgorithm | | | | |
| 3 | Create three strings with the different partyName (partyName1, partyName2, partyName3) | | | | |
| 4 | Create a vector of Party* (parties) | | | | |
| 5 | Create three different Party* with the using the respective strings from Step#2 for the constructor (party1, party2, party3) | | | | |
| 6 | Add all the parties to the vector created in Step #4 | | | | |
| 7 | Call the breakTiefunction and initialize it to a Party* | | | | |
| 8 | Test the equality of the parties created in step 5 | party == party1 \|\| party == party == party2 == party3 | true | true | |

**Post condition(s) for Test:**

The CPLAlgorithm breakTie function works as intended. After finding the winner of the tie, the algorithm can continue to calculate the seats and their respective seats.

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 41** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set CPLAlgorithmTest and the name of the test is the**

**distributeSeatsRandomly. This test ensures that the CPLAlgorithm can correctly distribute the seats randomly when need be.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of IRAlgorithm The creation of a CPLAlgoritm instance. It can be a default or overridden constructor. This test is only true when a party wins more seats than the number of candidates they have,**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a vector of Party* (parties) | | | | |
| 3 | Create three strings with the different partyName (partyName1, partyName2, partyName3) | | | | |
| 5 | Create three different Party* with the using the respective strings from Step#2 for the constructor (party1, party2, party3) | | | | |
| 6 | Create a vector string of canindaes (canindates) | | | | |
| 7 | Set all three parties candidates to the vector created in step 6 | | | | |
| 8 | Add all the parties to the parties vector created in step #2 | | | | |
| 9 | Create a vector of BallotCPl ballots | | | | |
| 10 | Create a bunch of BallotCPL instances with the corresponding Partyname and party instance | | | | |
| 11 | Add all of the ballots created | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | in Step #10 to the ballots vector created in Step#9 | | | | |
| 12 | Create an instance of a CPL algorithm with ballots, 3, parties, 2, 6 | | | | |
| 13 | Count the ballots | | | | |
| 14 | Distribute the seats randomly | | | | |
| 15 | Create an intiiazle a counter variable to 0 and the loop through all of the parties and update the count based on the seats received | | | | |
| 16 | Test the equality of the count | EXPECT_EQ(count, 1) | 1 | 1 | |
| 17 | Redistrivute seats randomly with input 2 | | | | |
| 18 | Step #15 | | | | |
| 16 | Test the equality of the count | EXPECT_EQ(count, 3) | 3 | 3 | |

**Post condition(s) for Test:**

The CPLAlgorithm distributeSeatsRandomly function works as intended.

# PARTY UNIT TEST

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 42** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the seats is emptyConstructur This test ensures that the Party class empty constructor works with the correct default values .**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | Create an party instance with an empty constructor | | | | |
| 3 | Create a vector of string that contains the expected candidates | | | | |
| 5 | Create and initialize two integer varaibels with the integer 0. (expected remainingVotes and expectedSeatsRecieved) | | | | |
| 6 | Create a vector string of canindaes (canindates) | | | | |
| 7 | Test the equality of the private variables of the default constructor | party.getCandiates()<br>party.getRemainingVotes()<br>party.getSeatsReecieed | vector<string><br>0<br>0 | vector<string><br>0<br>0 | |

**Post condition(s) for Test:**

The empty constructor correctly sets up the private variables correctly, and can be correctly used through the election class.

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 43** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the seats is emptyConstructur This test ensures that the Party class empty constructor works with the correct default values .**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | Create a string name with default (“dems”) | | | | |
| 3 | Create an integer and initializes the expected remaining number of votes (expectedRemainingVotes) | | | | |
| 5 | Create a vector of string that represents the expected Candidates | | | | |
| 6 | Create an integer that represents the expectedSeatsReceived | | | | |
| 7 | Create a party with the overridden Party constructor and party name | | | | |
| 8 | Set the remainingvotes to the expectedRemainingVotes and the addSeatsReceived should be equal to values provided | party.getCandiates()<br>party.getRemainingVotes()<br>party.getSeatsRecieved() | vector<string><br>10<br>5 | vector<string><br>10<br>5 | |

**Post condition(s) for Test:**

The Party constructor correctly initializes the correct values for the private variables based on inputs from the Constructor

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __** **Test Date: March 25, 2023**

**Test Case ID#: 44** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the test is getName. This test ensures that the getter works correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create a string name with default testing value (“dems”) | | | | |
| 3 | Create an instance of the Party with the string created from Step #2 | | | | |
| 4 | Test the partyName is equal to the string value provided in step #2 | party.getName() | “dems” | “dems” | |

**Post condition(s) for Test:**

The Party class with the getName works correctly

## Project Name: Project 1: Voting System

**Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 45**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the test is an increment. This test ensures that the increment vote works correctly for the Party Class**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an empty constructor of the Party class | | | | |
| 3 | check the remaining Votes of the party | party.getRemainingVotes() | 0 | 0 | |
| 4 | Increment the party vote | | | | |
| 5 | check the remaining Votes of the party | party.getRemainingVotes() | 1 | 1 | |
| 6 | Increment the party vote | | | | |
| 7 | check the remaining Votes of the party | party.getRemainingVotes() | 2 | 2 | |
| 8 | Increment the party vote | | | | |

**Post condition(s) for Test:**

The Party class incrementVote function works correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 46**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the test is getAndSetRemiaingVotes. This test ensures that the getandSetter works correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | Create an empty constructor of the Party class | | | | |
| 3 | Create an integer that represents the number of votes(votes) | | | | |
| 4 | Set the remainingVotes of the party to the integer created in Step #3 | | | | |
| 8 | Check the equality of the remaining votes and the step #3 | party.getRemainingVotes() | 832 | 832 | |

**Post condition(s) for Test:**

The Party class getRemainingVotes function works correctly

## Project Name: Project 1: Voting System Team# 8

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 47**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the test is getSeatsRemaining. This test ensures that the getSeatsRemaing function works correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes_X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an empty constructor of the Party class | | | | |
| 3 | Create a vector of string candidates that contain candidate names | | | | |
| 4 | Sets the candidates of the party to the list of candidates | | | | |
| 5 | Add the seatsRecieved to the party | | | | |
| 8 | Test the equality of getSeatsRemainig | party.getSeatsRemaining() | 1 | 1 | |

**Post condition(s) for Test:**

The Party class getSeatsRemaining() function works correctly

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 47**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is PartySet and the name of the test is getandSetCandidates. This test ensures that the getters and setters work correctly for the candidates**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X__ no ___**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Party**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | Create an empty constructor of the Party class | | | | |
| 3 | Create a vector of string that contains dummy candidates | | | | |
| 4 | Set the candidates of the party class to the vector created in #3 | | | | |
| 5 | Test the equality of party.getCandidates() | party.candidates() | candidates | candidates | |

**Post condition(s) for Test:**

The Party class getter and setter works correctly for the candidate

# Candidate Unit TESTS

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit _X_ System __**

**Test Date: March 25, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: The name of the testing set is CandidateTest and the name of the seats is fullTest. This test ensures that the Candidate class constructor, getter and setter work correctly**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes X no**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make test and ./Test of Candidate**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | create candidate object | | | | |
| 3 | check if candidate ballot is empty | candidate.getBallots().empty() == true | true | true | |
| 5 | check if candidate is active | candidate.getActive() == true | true | true | |
| 6 | check if the candidate total vote is 0 | candidate.getCandidateTotalVote() == 0 | true | true | |
| 7 | check if the candidate name is “” | candidate.getCandidateName() == "" | true | true | |
| 8 | set the candidate ‘s active to be false | | | | |
| 9 | check if candidate is active | | | | |
| 10 | create 2 ballots and add them to Candidate | | | | |
| 11 | check if candidate have the first ballot | candidate.getBallots().at(0)->getCandidate(0) == "Test", | true | true | |
| 12 | check candidate total vote is 1 | candidate.getCandidateTotalVote() == 1 | true | true | |
| 13 | check if the candiade have the second ballot | candidate.getBallots().at(1)->getCandidate(0) == "Test1" | true | true | |
| 14 | check if the candidate totla vote is 2 | candidate.getCandidateTotalVote() == 2 | true | true | |
| 15 | create a candidate with | | | | |

| | | | | | |
|---|---|---|---|---|---|
| | parameter | | | | |
| 16 | check the getBallot | candidate2.getBallots().empty() == true | true | true | |
| 17 | check the get active | candidate2.getActive() == true | true | true | |
| 18 | check the get candidate total vote | candidate2.getCandidateTotalVote() == 0 | true | true | |
| 19 | check the get candidate name | candidate2.getCandidateName() == "Test" | true | true | |
| 20 | set the candidate 2 active to be false | | | | |
| 21 | check candidate 2 active | candidate2.getActive() == false | true | true | |

**Post condition(s) for Test:**

The candidate class work correctly for constructor, getter and setter

# IR SYSTEM TESTS

**Project Name: Project 1: IR System Tests** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 42** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing when all ballots are the same vote.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**When running make has to be successful in order to test the SystemTestIR1.csv file.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | In src folder run make. | | | | |
| 3 | the run ./run SystemTestIR1.csv | | In terminal it should print:<br>IR Results:<br>Number of ballots: 5<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 0 votes, (0% of the total votes)<br>Chou(I) recieved 5 votes, (100% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Chou(I) | In terminal it actually prints:<br>IR Results:<br>Number of ballots: 5<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 0 votes, (0% of the total votes)<br>Chou(I) recieved 5 votes, (100% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Chou(I) | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: IR System Tests** **Team# 8**

**Test Stage: Unit __ System _X_**

**Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing when all ballots are the same vote.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass _____ Fail___X_____**

**Preconditions for Test:**
**When running make has to be successful in order to test the SystemTestIR2.csv file.**

| Step | Test Step | Test | Expected | Actual | |
|---|---|---|---|---|---|

| # | Description | Data | Result | Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | In src folder run make. | | | | |
| 3 | the run ./run SystemTestIR2.csv | | In terminal it should print:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 4<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 2 votes, (50% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Kleinberg(R) | In terminal it actually prints:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 4<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 2 votes, (50% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Kleinberg(R) | It will occasiablly print the winning candidate as Chou(I) since sometimes it will redistribute the ballots Rosen, Kleinberg and Royce first. If it manages to redistribute Chou since the pick is random since they all are the least voted then, it will work as intended |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: IR System Tests** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 44** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing with ballots that have same number of votes and so it should either result in Rosen or Kleinberg as the winning candidate.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes___ no __X_**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**When running make has to be successful in order to test the SystemTestIR3.csv file.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | In src folder run make. | | | | |
| 3 | the run ./run SystemTestIR3.csv | | In terminal it should print:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 6<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 2 votes, (33% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Kleinberg(R)<br><br>or<br><br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 4<br>Number of candidates: 4<br>Rosen(D) recieved 2 votes, (33% of the total votes)<br>Kleinberg(R) recieved 0 votes, (0% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Rosen(D) | In terminal it actually prints:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 6<br>Number of candidates: 4<br>Rosen(D) recieved 0 votes, (0% of the total votes)<br>Kleinberg(R) recieved 2 votes, (33% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Kleinberg(R)<br><br>or<br><br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 4<br>Number of candidates: 4<br>Rosen(D) recieved 2 votes, (33% of the total votes)<br>Kleinberg(R) recieved 0 votes, (0% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 0 votes, (0% of the total votes)<br><br>Winning Candidate: Rosen(D) | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: IR System Tests** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 45** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing with lots of ballot where the winner would be Royce**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass X Fail**

**Preconditions for Test:**
**When running make has to be successful in order to test the SystemTestIR4.csv file.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | In src folder run make. | | | | |
| 3 | the run ./run SystemTestIR4.csv | | In terminal it should print:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 13<br>Number of candidates: 4<br>Rosen(D) recieved 1 votes, (7% of the total votes)<br>Kleinberg(R) recieved 5 votes, (38% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 7 votes, (53% of the total votes)<br><br>Winning candidate: Royce(L) | In terminal it actually prints:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 13<br>Number of candidates: 4<br>Rosen(D) recieved 1 votes, (7% of the total votes)<br>Kleinberg(R) recieved 5 votes, (38% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 7 votes, (53% of the total votes)<br><br>Winning candidate: Royce(L) | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

## Project Name: Project 1: IR System Tests Team# 8

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 46** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing with lots of ballots and the winner is**

**is Royce.**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**When running make has to be successful in order to test the SystemTestIR5.csv file.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |
| 2 | In src folder run make. | | | | |
| 3 | the run ./run SystemTestIR5.csv | | In terminal it should print:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 13<br>Number of candidates: 4<br>Rosen(D) recieved 1 votes, (7% of the total votes)<br>Kleinberg(R) recieved 5 votes, (38% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 7 votes, (53% of the total votes)<br><br>Winning candidate: Royce(L) | In terminal it actually prints:<br>IR Results:<br>Date: 12-12-1212<br>Type: IR<br>Number of ballots: 13<br>Number of candidates: 4<br>Rosen(D) recieved 1 votes, (7% of the total votes)<br>Kleinberg(R) recieved 5 votes, (38% of the total votes)<br>Chou(I) recieved 0 votes, (0% of the total votes)<br>Royce(L) recieved 7 votes, (53% of the total votes)<br><br>Winning candidate: Royce(L) | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

# CPL SYSTEM TESTS

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 47** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL1.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL1.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL1.csv | | | | |
| | Fill in the date input 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 9<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 3 votes (33% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 2 votes (22% | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 9<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 3 votes (33% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 2 votes (22% of the | |

| | | | | | |
|---|---|---|---|---|---|
| | | | of the total votes) for candidates: Green<br>NewWave received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Reform received: 1 seats with 2 votes (22% of the total votes) for candidates: McClure<br>Green received: 0 seats with 1 votes (11% of the total votes) for candidates:<br>Independent received: 0 seats with 1 votes (11% of the total votes) for candidates: | total votes) for candidates: Green<br>NewWave received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Reform received: 1 seats with 2 votes (22% of the total votes) for candidates: McClure<br>Green received: 0 seats with 1 votes (11% of the total votes) for candidates:<br>Independent received: 0 seats with 1 votes (11% of the total votes) for candidates: | |
| | | | | | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 43** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL2.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes___ no _X__**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL2.csv to run this system test.**

| Step # | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 1 | | | | | |

| | | | | | |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL2.csv | | | | |
| | Fill in the date input: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 5<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 1 votes (20% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 1 votes (20% of the total votes) for candidates: Green<br>NewWave received: 1 seats with 1 votes (20% of the total votes) for candidates: Jacks<br>Reform received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Green received: 1 seats with 1 votes (20% of the total votes) for candidates: Zheng<br>Independent received: 1 seats with 1 votes (20% of the total votes) for candidates: Peters | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 5<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 1 votes (20% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 1 votes (20% of the total votes) for candidates: Green<br>NewWave received: 1 seats with 1 votes (20% of the total votes) for candidates: Jacks<br>Reform received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Green received: 1 seats with 1 votes (20% of the total votes) for candidates: Zheng<br>Independent received: 1 seats with 1 votes (20% of the total votes) for candidates: Peters | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 43** **Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL3.csv**

**Automated: yes___ no _X__**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL3.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL3.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 90<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 3 seats with 30 votes (33% of the total votes) for candidates: Foster Volz Pike<br>Republican received: 3 seats with 20 votes (22% of the total votes) for candidates: Green Xu Wang<br>NewWave received: 2 seats with 0 votes (0% of the total votes) for candidates: Jacks Rosen<br>Reform received: 2 seats with 20 votes (22% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 10 votes (11% of the total votes) for candidates: Zheng Melvin<br>Independent received: 1 seats with 10 votes (11% of the total votes) for candidates: Peters | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 90<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 3 seats with 30 votes (33% of the total votes) for candidates: Foster Volz Pike<br>Republican received: 3 seats with 20 votes (22% of the total votes) for candidates: Green Xu Wang<br>NewWave received: 2 seats with 0 votes (0% of the total votes) for candidates: Jacks Rosen<br>Reform received: 2 seats with 20 votes (22% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 10 votes (11% of the total votes) for candidates: Zheng Melvin<br>Independent received: 1 seats with 10 votes (11% of the total votes) for candidates: Peters | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Test Stage: Unit __ System _X_**

**Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL4.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL4.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL4.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 90<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 3 seats with 30 votes (33% of the total votes) for candidates: Foster Volz Pike<br>Republican received: 3 seats with 20 votes (22% of the total votes) for candidates: Green Xu Wang<br>NewWave received: 2 seats with 0 votes (0% of the total votes) for candidates: Jacks Rosen<br>Reform received: 2 seats with 20 votes (22% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 10 votes (11% of the total votes) for candidates: Zheng Melvin<br>Independent received: 1 seats with 10 votes (11% of the total votes) for candidates: Peters | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 90<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 3 seats with 30 votes (33% of the total votes) for candidates: Foster Volz Pike<br>Republican received: 3 seats with 20 votes (22% of the total votes) for candidates: Green Xu Wang<br>NewWave received: 2 seats with 0 votes (0% of the total votes) for candidates: Jacks Rosen<br>Reform received: 2 seats with 20 votes (22% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 10 votes (11% of the total votes) for candidates: Zheng Melvin<br>Independent received: 1 seats with 10 votes (11% of the total votes) for candidates: Peters | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date.

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_**

**Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL5.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL5.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL5.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 1<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Republican received: 0 seats with 0 votes (0% of the total votes) for candidates: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 1<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Republican received: 0 seats with 0 votes (0% of the total votes) for candidates: | |

| | | | | | |
|---|---|---|---|---|---|
| | | | NewWave received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Reform received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Green received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Independent received: 0 seats with 0 votes (0% of the total votes) for candidates: | NewWave received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Reform received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Green received: 0 seats with 0 votes (0% of the total votes) for candidates:<br>Independent received: 0 seats with 0 votes (0% of the total votes) for candidates: | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date. Test with no seat available

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_**

**Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL6.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL6.csv to run this system test.**

| Step | Test Step | Test | Expected | Actual | |
|---|---|---|---|---|---|

| # 1 | Description | Data | Result | Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL6.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 0<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Republican received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>NewWave received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Reform received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Green received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Independent received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 0<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Republican received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>NewWave received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Reform received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Green received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates:<br>Independent received: 0 seats with 0 votes (-2147483648% of the total votes) for candidates: | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date. Test with no ballot cast

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_**

**Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL7.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes ___ no X**

**Results: Pass __X___ Fail_____**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL7.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL7.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 236<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 2 seats with 32 votes (13% of the total votes) for candidates: Foster Volz<br>Republican received: 2 seats with 56 votes (23% of the total votes) for candidates: Green Xu<br>NewWave received: 1 seats with 32 votes (13% of the total votes) for candidates: Jacks<br>Reform received: 2 seats with 64 votes (27% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 48 votes (20% of the total votes) for candidates: Zheng Melvin<br>Independent received: 0 seats with 4 votes (1% of the total votes) for candidates: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 236<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 2 seats with 32 votes (13% of the total votes) for candidates: Foster Volz<br>Republican received: 2 seats with 56 votes (23% of the total votes) for candidates: Green Xu<br>NewWave received: 1 seats with 32 votes (13% of the total votes) for candidates: Jacks<br>Reform received: 2 seats with 64 votes (27% of the total votes) for candidates: McClure Berg<br>Green received: 2 seats with 48 votes (20% of the total votes) for candidates: Zheng Melvin<br>Independent received: 0 seats with 4 votes (1% of the total votes) for candidates: | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date. Test with no ballot cast

**Project Name: Project 1: Voting System** **Team# 8**

**Test Stage: Unit __ System _X_** **Test Date: March 26, 2023**

**Test Case ID#: 43**

**Name(s) of Testers: Bang, Aiden, Nuh, James**

**Test Description: Testing the system file with the SystemTestCPL8.csv**

**Indicate where are you storing the tests (what file) and the name of the method/functions being used.**

**Automated: yes no X**

**Results: Pass X Fail**

**Preconditions for Test:**
**A successful make in src folder and ./run SystemTestCPL8.csv to run this system test.**

| Step # 1 | Test Step Description | Test Data | Expected Result | Actual Result | Notes |
|---|---|---|---|---|---|
| 2 | In src folder run make. | | | | |
| | the run ./run SystemTestCPL8.csv | | | | |
| | Fill in the date: 12-12-2022 | In terminal it should print: | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 126<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 21 votes (16% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 21 votes (16% of the total votes) for candidates: Green<br>NewWave received: 1 seats with 21 votes (16% of the total votes) for candidates: Jacks<br>Reform received: 1 seats with 21 votes (16% of the total votes) for candidates: McClure<br>Green received: 1 seats with 21 votes (16% of the total votes) for candidates: Zheng<br>Independent received: 1 seats with 21 votes (16% of the total votes) for candidates: Peters | CPL Results:<br>Date: 12-12-2022<br>Type: CPL<br>Number of ballots cast: 126<br>Number of Parties: 6<br>Winning candidates and their parties<br>Democratic received: 1 seats with 21 votes (16% of the total votes) for candidates: Foster<br>Republican received: 1 seats with 21 votes (16% of the total votes) for candidates: Green<br>NewWave received: 1 seats with 21 votes (16% of the total votes) for candidates: Jacks<br>Reform received: 1 seats with 21 votes (16% of the total votes) for candidates: McClure<br>Green received: 1 seats with 21 votes (16% of the total votes) for candidates: Zheng<br>Independent received: 1 seats with 21 votes (16% of the total votes) for candidates: Peters | |

**Post condition(s) for Test:**

After the test is run it will mainly just print out results to terminal and then create an audit file based on input date. Test with even number of vote for each party

**Project Name:** The project #, name of your system, and the team#

**Test Stage:** Indicate whether it is a unit test or a system test.

**Test Date:** The date the test was performed.

**Test Case ID#:** A unique ID is required. Decide on a naming convention and use numbering. Example: Ballot_Shuffle_1

**Name(s) of Testers:** List the names of anyone involved in running this test case.

**Test Description:** Describe briefly the test objective.

**Automated:** Indicate if the test is completely automated or being checked manually. (If you have methods running the tests and checking results, select "yes". If you are manually checking results, indicate manual by selecting the "no.")

**Results:** Indicate if the test passed or failed.

**Step #:** You will be listing the test steps in order. This number is the step number in the process.

**Test Step Description:** Details of the test step.

**Test Data:** What the test data will be for this step. Be clear on what the input data will be. If using a specific file, be clear on the name.

**Expected Result:** What result are you expecting from the program component or system.

**Actual Result:** What result were returned based on the test.

**Post condition for Test:** What will be true after the test has been run?  Has the state of the system changed in any way?

**Notes:**  Comments and notes for you and your team members.